Bazy Danych – Projekt

Etap 2: projektowanie

Aplikacja do zarządzania budżetem domowym

Jan Nowak 284138, Kacper Kobylarczyk 284167

Prowadzący: mgr inż. Michał Jaroszczuk
Grupa zajęciowa: 2

Repozytorium: `https://github.com/melamann00/bazy-danych-projekt`

Wrocław, 24 października 2026

Spis treści

1 Wprowadzenie **3**
1.1 Tabela kryteriów Etapu 2 . . . 3

2 Wybrane technologie **3**
2.1 Zestawienie . . . 3
2.2 Architektura . . . 4
2.3 Wybór SQLite i ograniczenia . . . 4
2.4 Konfiguracja bazy w Django . . . 5

3 Projekt bazy danych **5**
3.1 Konwencje i założenia . . . 5
3.2 Diagram ERD . . . 5
3.3 Relacje między tabelami . . . 6
3.4 Słownik danych (opis tabel) . . . 8
3.5 Normalizacja . . . 13
3.6 Indeksy . . . 14
3.7 Widoki, wyzwalacze i procedury . . . 14

4 Transakcje i zapytania SQL **15**
4.1 Transakcje modyfikujące dane . . . 15
4.1.1 T1 – Rejestracja wydatku lub przychodu . . . 15
4.1.2 T2 – Edycja transakcji . . . 15
4.1.3 T3 – Usunięcie transakcji (usunięcie logiczne) . . . 16
4.1.4 T4 – Przelew między kontami . . . 16
4.1.5 T5 – Zaproszenie, akceptacja i usunięcie członka gospodarstwa . . . 16
4.1.6 T6 – Ustawienie lub zmiana limitu miesięcznego . . . 16
4.1.7 T7 – Generowanie transakcji cyklicznych . . . 16
4.1.8 T8 – Wpłata na cel oszczędnościowy . . . 17
4.2 Zapytania odczytujące (zaawansowane) . . . 17
4.2.1 Q1 – Wydatki wg kategorii w wybranym okresie . . . 17
4.2.2 Q2 – Plan a wykonanie: kategorie w strefie ostrzeżenia lub po przekroczeniu limitu . . . 17
4.2.3 Q3 – Bilans miesiąca . . . 17
4.2.4 Q4 – Wydatki wg osoby z udziałem procentowym (funkcja okna) . . . 17
4.2.5 Q5 – Trend miesięczny wydatków (CTE i funkcja LAG) . . . 17
4.2.6 Q6 – Transakcje z tagami . . . 18
4.2.7 Q7 – Widoki: wykorzystanie limitów, postęp celów, weryfikacja sald . . . 18

5 Prognoza charakteru encji **18**

6 Mapowanie wymagań biznesowych na wymagania funkcjonalne i niefunkcjo-

nalne **19**

7 Diagramy UML **21**

7.1 Diagram aktywności: rejestracja wydatku 21

7.2 Diagram sekwencji: rejestracja wydatku 22

8 Opisy przypadków użycia **23**

8.1 UC.04 Rejestracja transakcji (przychód/wydatek) 23

8.2 UC.05 Przelew między kontami 23

8.3 UC.06 Ustalanie limitu miesięcznego 24

8.4 UC.07 Zarządzanie regułami cyklicznymi 24

8.5 UC.08 Generowanie transakcji cyklicznych 25

8.6 UC.09 Cel oszczędnościowy i wpłaty 25

8.7 UC.10 Raport miesięczny 26

8.8 UC.11 Zapraszanie członków do gospodarstwa 26

9 Podsumowanie **27**

1 Wprowadzenie

Dokument stanowi realizację Etapu 2 projektu z przedmiotu Bazy Danych. Rozwija zakres zdefiniowany w Etapie 1 i przekłada go na decyzje techniczne: wybór technologii, projekt bazy danych, opis transakcji i zapytań SQL oraz zestaw artefaktów analitycznych (wymagania funkcjonalne i niefunkcjonalne, diagramy UML, opisy przypadków użycia).

1.1 Tabela kryteriów Etapu 2

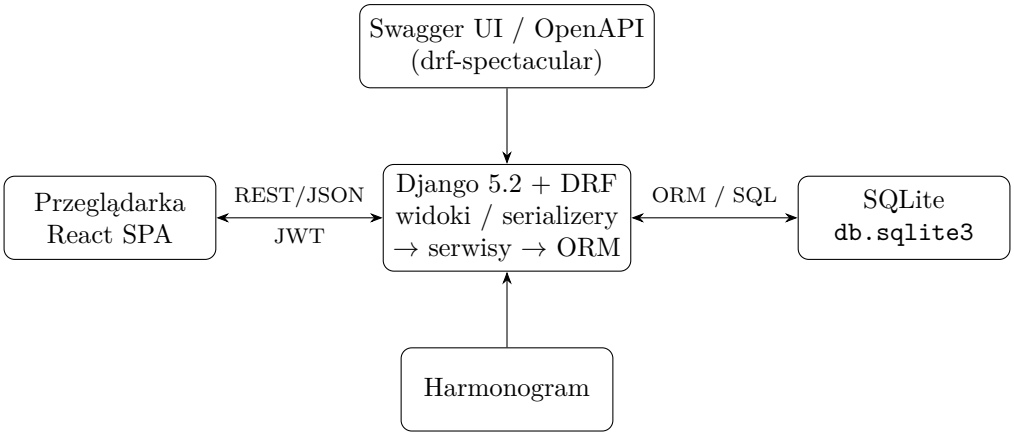
Wymagany element	**Rozdział**
Typ aplikacji, typ bazy danych, technologie backend i frontend	2
Projekt bazy danych: encje, relacje, diagram ERD, opis tabel, normalizacja	3
Opis transakcji i zapytania SQL, w tym ponad 3 zaawansowane	4
Prognoza charakteru encji (zapis/odczyt, zmienność, liczność)	5
Własności bazy: $\geq$5 tabel, relacje 1:1, 1:n, n:m, 3NF, pola audytowe, więzy, JOIN/GROUP BY/HAVING, widoki, procedury	3, 4
Mapowanie WB $\rightarrow$ WF/WNF	6
Diagramy: przypadków użycia, aktywności, maszyny stanowej, sekwencji	7
Opisy przypadków użycia (aktorzy, ścieżka podstawowa i alternatywne)	8

2 Wybrane technologie

2.1 Zestawienie

Typ aplikacji	Aplikacja webowa: jednostronicowy frontend (SPA) komunikujący się z backendem przez REST API (JSON). Tylko wersja przeglądarkowa.
Typ bazy danych	SQLite 3 (relacyjna, plikowa, wbudowana w Pythona), tryb WAL, włączone klucze obce.
Backend	Python 3.12+, Django 5.2 LTS (ORM, migracje, uwierzytelnianie, panel administracyjny), Django REST Framework (serializery, widoki API), djangorestframework-simplejwt (tokeny JWT), django-filter (filtrowanie list), drf-spectacular (OpenAPI/Swagger).
Frontend	React z narzędziem budowania Vite, TanStack Query (komunikacja z API i cache), Recharts (wykresy raportów).
Testy	pytest-django (testy jednostkowe i integracyjne backendu), Playwright (testy interfejsu, Etap 4).
Narzędzia	Git i GitHub, Graphviz i TikZ (diagramy), LaTeX (dokumentacja), Postman/Swagger UI.

2.2 Architektura



Rysunek 1: Architektura systemu

2.3 Wybór SQLite i ograniczenia

SQLite jest najprostszym wyborem dla projektu: nie wymaga serwera, jest obsługiwana przez Django bez dodatkowej konfiguracji, a plik bazy można łatwo skopiować i wersjonować danymi testowymi. Docelowe obciążenie (rozdz. 5: ok. 10 gospodarstw, 60 tys. transakcji) mieści się w możliwościach SQLite przy poprawnych indeksach. Świadomie uwzględniamy ograniczenia tego silnika i sposób ich obejścia:

- **Brak typu DECIMAL** – kwoty przechowujemy jako `INTEGER` w groszach, więc sumy w SQL są dokładne.
- **Brak procedur składowanych** – ich rolę pełnią funkcje warstwy serwisowej wykonywane w jednej transakcji (`transaction.atomic`), a niezmienniki między tabelami wymuszają wyzwalacze (rozdz. 3.7). Zastąpienie procedur składowanych jest jednym z elementów, które regulamin pozwala uzgodnić z prowadzącym.
- **Jeden pisarz naraz** – zapisy są serializowane. Transakcje zapisujące uruchamiamy w trybie `BEGIN IMMEDIATE`, aby uniknąć konfliktów przy awansie blokady, a tryb WAL pozwala odczytom działać równolegle z zapisem.
- **Ograniczone ALTER TABLE** – zmiany schematu wykonuje mechanizm migracji Django.
- **Możliwość migracji** – ponieważ dostęp do danych odbywa się przez ORM, zmiana silnika na PostgreSQL wymaga głównie zmiany konfiguracji (i przeniesienia widoków/wyzwalaczy).

2.4 Konfiguracja bazy w Django

```
DATABASES = {
  "default": {
      "ENGINE": "django.db.backends.sqlite3",
      "NAME": BASE_DIR / "db.sqlite3",
      "OPTIONS": {
          "transaction_mode": "IMMEDIATE",
          "init_command": "PRAGMA journal_mode=WAL; PRAGMA synchronous=NORMAL;",
          "timeout": 10,
      },
  }
}
AUTH_USER_MODEL = "budget.User"  # własny model użytkownika
```

3 Projekt bazy danych

3.1 Konwencje i założenia

- **Klucze główne** – sztuczne, całkowitoliczbowe (`id`, autoinkrementacja). Unikalność naturalną (np. nazwa konta w gospodarstwie) wymuszają osobne ograniczenia `UNIQUE`.
- **Pola audytowe** – każda tabela encji ma `created_at`, `updated_at` i `is_active`. Usunięcie ma charakter logiczny (`is_active = 0`), co zachowuje historię finansową i spójność kluczy obcych. Wyjątkiem jest tabela łącząca `fin_transaction_tag`, będąca czystą relacją n:m, czyli jej wiersze znikają razem z transakcją lub tagiem.
- **Kwoty** – liczby całkowite w groszach, kolumny z sufiksem `_gr`. Kwota transakcji jest zawsze dodatnia, a znak wynika z typu operacji.
- **Daty** – `DATE` w formacie RRRR-MM-DD. Zakresy dat zapisujemy jako przedziały półotwarte (`>=` początek i `<` koniec), co ułatwia użycie indeksów.
- **Enumeracje** – kolumny tekstowe z ograniczeniem `CHECK ... IN (...)`.

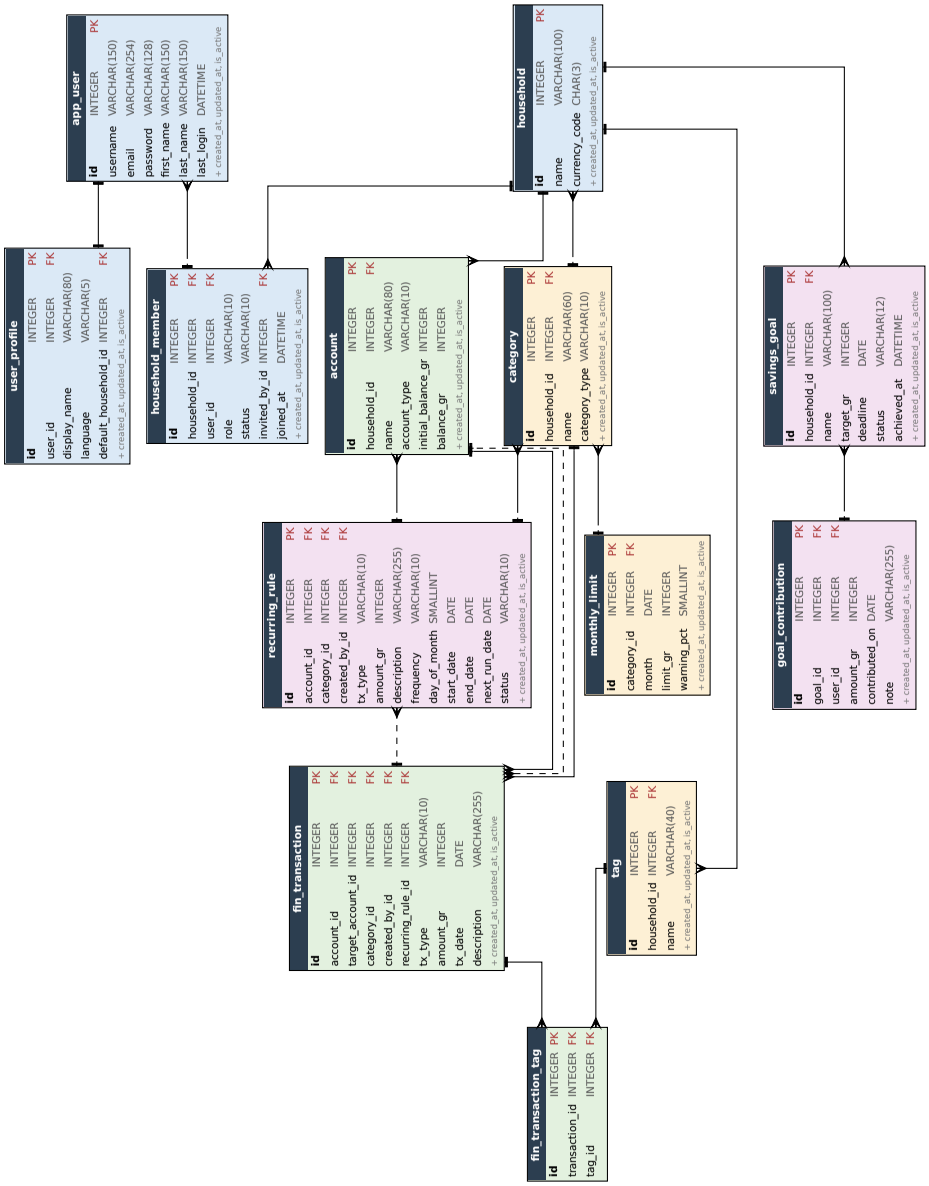
3.2 Diagram ERD

Diagram (rys. 2) przedstawia 13 tabel, ich kolumny i relacje. Dla czytelności pominięto krawędzie do `app_user` z kolumn `created_by_id`, `invited_by_id`, `goal_contribution.user_id` oraz z `user_profile.default_household_id` do `household`. Wszystkie relacje wypisano w tabeli 3.

3.3 Relacje między tabelami

Tabela 3: Wykaz relacji

Tabela (strona „wiele”)	Tabela (strona „jeden”)	Typ	Znaczenie
`user_profile.user_id`	`app_user`	1:1	ustawienia użytkownika (`UNIQUE`)
`household_member`	`app_user`, `household`	n:m	użytkownik w wielu gospodarstwach, gospodarstwo z wieloma użytkownikami (z rolą)
`fin_transaction_tag`	`fin_transaction`, `tag`	n:m	dowolne tagi przy transakcji
`account`, `category`, `tag`, `savings_goal`	`household`	1:n	dane należą do gospodarstwa
`fin_transaction.account_id`	`account`	1:n	konto, którego dotyczy operacja (przy przelewie źródłowe)
`fin_transaction.target_account_id`	`account`	1:n (opcj.)	konto docelowe przelewu
`fin_transaction.category_id`	`category`	1:n (opcj.)	kategoria klasyfikująca (przelew jej nie ma)
`fin_transaction.created_by_id`	`app_user`	1:n	kto wprowadził transakcję
`fin_transaction.recurring_rule_id`	`recurring_rule`	1:n (opcj.)	reguła, która wygenerowała transakcję
`recurring_rule`	`account`, `category`, `app_user`	1:n	składniki szablonu transakcji i autor
`monthly_limit.category_id`	`category`	1:n	limity miesięczne kategorii
`goal_contribution.goal_id`	`savings_goal`	1:n	wpłaty na cel
`goal_contribution.user_id`	`app_user`	1:n	kto wpłacił
`household_member.invited_by_id`	`app_user`	1:n (opcj.)	kto zaprosił
`user_profile.default_household_id`	`household`	1:n (opcj.)	domyślne gospodarstwo



Rysunek 2: Diagram ERD (niebieski: użytkownicy i gospodarstwa, zielony: konta i transakcje, żółty: słowniki i plan, fioletowy: reguły cykliczne i cele)

3.4 Słownik danych (opis tabel)

Poniżej opisano przeznaczenie każdej tabeli oraz jej kolumny. Pola audytowe są wspólne dla wszystkich tabel z wyjątkiem `fin_transaction_tag`. W kolumnie ograniczeń „FK→” oznacza klucz obcy.

1. app_user

Użytkownik systemu. Przechowuje dane logowania i podstawowe dane osobowe. Użytkownik może należeć do wielu gospodarstw.

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`username`	VARCHAR(150)	NOT NULL UNIQUE	login
`email`	VARCHAR(254)	NOT NULL UNIQUE	adres e-mail
`password`	VARCHAR(128)	NOT NULL	skrót hasła (PBKDF2/Argon2)
`first_name`	VARCHAR(150)	NOT NULL DEFAULT ”	imię
`last_name`	VARCHAR(150)	NOT NULL DEFAULT ”	nazwisko
`last_login`	DATETIME	NULL	ostatnie logowanie
+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)			

2. household

Gospodarstwo domowe – grupa użytkowników współdzielących budżet. Jest korzeniem danych: konta, kategorie, tagi i cele należą do gospodarstwa.

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`name`	VARCHAR(100)	NOT NULL	nazwa gospodarstwa
`currency_code`	CHAR(3)	NOT NULL DEFAULT ’PLN’	waluta gospodarstwa (jedna na gospodarstwo)
+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)			

3. user_profile

Ustawienia użytkownika. Wydzielone, aby nie mieszać danych uwierzytelniania z preferencjami.

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`user_id`	INTEGER	NOT NULL UNIQUE FK→ `app_user(id)`	FK do użytkownika (UNIQUE = 1:1)
`display_name`	VARCHAR(80)	NOT NULL	nazwa wyświetlana
`language`	VARCHAR(5)	NOT NULL DEFAULT ’pl’	język interfejsu
`default_household_id`	INTEGER	NULL FK→ `household(id)`	domyślnie wybrane gospodarstwo
+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)			

4. household_member

Tabela łącząca użytkowników z gospodarstwami (n:m) wraz z rolą i statusem zaproszenia.

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`household_id`	INTEGER	NOT NULL FK→ `household(id)`	FK do gospodarstwa
`user_id`	INTEGER	NOT NULL FK→ `app_user(id)`	FK do użytkownika
`role`	VARCHAR(10)	NOT NULL DEFAULT 'member' `CHECK (role IN ('admin','member'))`	rola: administrator lub zwykły członek
`status`	VARCHAR(10)	NOT NULL DEFAULT 'invited' `CHECK (status IN ('invited','active'))`	status: zaproszony / aktywny
`invited_by_id`	INTEGER	NULL FK→ `app_user(id)`	kto zaprosił
`joined_at`	DATETIME	NULL	data akceptacji zaproszenia

+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)

```
UNIQUE (household_id, user_id)
CHECK (status = 'invited' OR joined_at IS NOT NULL)
```

5. account

Konto finansowe gospodarstwa (gotówka, rachunek bankowy, oszczędności) wraz z saldem. Saldo jest kontrolowaną denormalizacją (patrz rozdz. 3.5).

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`household_id`	INTEGER	NOT NULL FK→ `household(id)`	FK do gospodarstwa
`name`	VARCHAR(80)	NOT NULL	nazwa konta
`account_type`	VARCHAR(10)	NOT NULL `CHECK (account_type IN ('cash','bank','savings'))`	typ: gotówka / bank / oszczędności
`initial_balance_gr`	INTEGER	NOT NULL DEFAULT 0	saldo początkowe w groszach
`balance_gr`	INTEGER	NOT NULL DEFAULT 0	bieżące saldo w groszach (aktualizowane transakcyjnie)

+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)

```
UNIQUE (household_id, name)
```

6. category

Słownik kategorii przychodów i wydatków w obrębie gospodarstwa (np. Jedzenie, Transport, Wynagrodzenie).

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`household_id`	INTEGER	NOT NULL FK→ `household(id)`	FK do gospodarstwa
`name`	VARCHAR(60)	NOT NULL	nazwa kategorii
`category_type`	VARCHAR(10)	NOT NULL `CHECK (category_type IN ('income','expense'))`	typ: przychód / wydatek

+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)

```
UNIQUE (household_id, name, category_type)
```

7. recurring_rule

Reguła cykliczna – szablon, na podstawie którego system generuje powtarzalne transakcje (czynsz, abonamenty, raty).

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`account_id`	INTEGER	NOT NULL FK→ `account(id)`	FK do konta
`category_id`	INTEGER	NOT NULL FK→ `category(id)`	FK do kategorii
`created_by_id`	INTEGER	NOT NULL FK→ `app_user(id)`	autor reguły
`tx_type`	VARCHAR(10)	NOT NULL `CHECK (tx_type IN ('income','expense'))`	typ generowanej transakcji
`amount_gr`	INTEGER	NOT NULL `CHECK (amount_gr > 0)`	kwota w groszach
`description`	VARCHAR(255)	NOT NULL DEFAULT ”	opis
`frequency`	VARCHAR(10)	NOT NULL `CHECK (frequency IN ('weekly','monthly','yearly'))`	częstotliwość
`day_of_month`	SMALLINT	NULL `CHECK (day_of_month BETWEEN 1 AND 31)`	dzień miesiąca (dla monthly/yearly)
`start_date`	DATE	NOT NULL	data pierwszego wystąpienia
`end_date`	DATE	NULL	data zakończenia (opcjonalna)
`next_run_date`	DATE	NOT NULL	termin najbliższego generowania
`status`	VARCHAR(10)	NOT NULL DEFAULT 'active' `CHECK (status IN ('active','paused'))`	aktywna / wstrzymana

+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)

```
CHECK (frequency = 'weekly' OR day_of_month IS NOT NULL)
CHECK (end_date IS NULL OR end_date >= start_date)
```

8. fin_transaction

Pojedyncza operacja finansowa: przychód, wydatek lub przelew między kontami. Największa tabela systemu. Przelew ma konto docelowe i nie ma kategorii.

Kolumna	Typ	Ograniczenia	Opis
id	INTEGER	PK	klucz główny (sztuczny)
account_id	INTEGER	NOT NULL FK→ account(id)	konto (dla przelewu: źródłowe)
target_account_id	INTEGER	NULL FK→ account(id)	konto docelowe (tylko przelew)
category_id	INTEGER	NULL FK→ category(id)	kategoria (nie dla przelewu)
created_by_id	INTEGER	NOT NULL FK→ app_user(id)	kto wprowadził transakcję
recurring_rule_id	INTEGER	NULL FK→ recurring_rule(id)	reguła, która wygenerowała transakcję
tx_type	VARCHAR(10)	NOT NULL CHECK (tx_type IN ('income','expense','transfer'))	typ operacji
amount_gr	INTEGER	NOT NULL CHECK (amount_gr > 0)	kwota w groszach (zawsze dodatnia)
tx_date	DATE	NOT NULL	data operacji
description	VARCHAR(255)	NOT NULL DEFAULT ''	opis

+ pola audytowe: created_at, updated_at, is_active (patrz rozdz. 3.1)

```
CHECK ((tx_type = 'transfer' AND target_account_id IS NOT NULL AND category_id
IS NULL AND target_account_id <> account_id) OR (tx_type <> 'transfer' AND
target_account_id IS NULL AND category_id IS NOT NULL))
UNIQUE (recurring_rule_id, tx_date)
```

9. tag

Dodatkowa etykieta pozwalająca dowolnie grupować transakcje.

Kolumna	Typ	Ograniczenia	Opis
id	INTEGER	PK	klucz główny (sztuczny)
household_id	INTEGER	NOT NULL FK→ household(id)	FK do gospodarstwa
name	VARCHAR(40)	NOT NULL	nazwa tagu

+ pola audytowe: created_at, updated_at, is_active (patrz rozdz. 3.1)

```
UNIQUE (household_id, name)
```

10. fin_transaction_tag

Tabela łącząca transakcje z tagami (n:m).

Kolumna	Typ	Ograniczenia	Opis
id	INTEGER	PK	klucz główny (sztuczny)
transaction_id	INTEGER	NOT NULL FK→ fin_transaction(id)	FK do transakcji
tag_id	INTEGER	NOT NULL FK→ tag(id)	FK do tagu

```
UNIQUE (transaction_id, tag_id)
```

11. monthly_limit

Planowana maksymalna kwota wydatków dla kategorii w danym miesiącu (jeden limit na parę kategoria–miesiąc).

Kolumna	Typ	Ograniczenia	Opis
id	INTEGER	PK	klucz główny (sztuczny)
category_id	INTEGER	NOT NULL FK→ category(id)	FK do kategorii wydatków
month	DATE	NOT NULL CHECK (strftime('%d', month) = '01')	pierwszy dzień miesiąca
limit_gr	INTEGER	NOT NULL CHECK (limit_gr > 0)	limit w groszach
warning_pct	SMALLINT	NOT NULL DEFAULT 80 CHECK (warning_pct BETWEEN 1 AND 100)	próg ostrzeżenia w procentach

\+ pola audytowe: created_at, updated_at, is_active (patrz rozdz. 3.1)

```
UNIQUE (category_id, month)
```

12. savings_goal

Cel oszczędnościowy: kwota docelowa, termin i status realizacji. Zgromadzona kwota jest wyliczana z wpłat.

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`household_id`	INTEGER	NOT NULL FK→ `household(id)`	FK do gospodarstwa
`name`	VARCHAR(100)	NOT NULL	nazwa celu
`target_gr`	INTEGER	NOT NULL `CHECK (target_gr > 0)`	kwota docelowa w groszach
`deadline`	DATE	NOT NULL	termin realizacji
`status`	VARCHAR(12)	NOT NULL DEFAULT `'in_progress'` `CHECK (status IN ('in_progress','achieved','cancelled'))`	status celu
`achieved_at`	DATETIME	NULL	data osiągnięcia celu

+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)

```
CHECK ((status = 'achieved') = (achieved_at IS NOT NULL))
```

13. goal_contribution

Pojedyncza wpłata na cel oszczędnościowy (rezerwacja środków; nie zmienia sald kont).

Kolumna	Typ	Ograniczenia	Opis
`id`	INTEGER	PK	klucz główny (sztuczny)
`goal_id`	INTEGER	NOT NULL FK→ `savings_goal(id)`	FK do celu
`user_id`	INTEGER	NOT NULL FK→ `app_user(id)`	kto wpłacił
`amount_gr`	INTEGER	NOT NULL `CHECK (amount_gr > 0)`	kwota w groszach
`contributed_on`	DATE	NOT NULL	data wpłaty
`note`	VARCHAR(255)	NOT NULL DEFAULT ”	notatka

+ pola audytowe: `created_at, updated_at, is_active` (patrz rozdz. 3.1)

3.5 Normalizacja

1NF. Wszystkie kolumny są atomowe, nie ma powtarzających się grup. Tagi transakcji, wyniesiono do tabeli `tag` i tabeli łączącej `fin_transaction_tag`. Podobnie członkostwo w gospodarstwie jest osobną tabelą `household_member`.

2NF. Każda tabela ma jednokolumnowy klucz główny `id`, więc zależności częściowe od części klucza złożonego nie występują. Klucze naturalne są zabezpieczone `UNIQUE`.

3NF. Atrybuty niekluczowe zależą wyłącznie od klucza. Aby uniknąć zależności przechodnich, nie powielamy danych dostępnych przez klucze obce:

- limit miesięczny nie przechowuje gospodarstwa, bo wynika z kategorii,
- nazwy kategorii, kont i użytkowników występują tylko w swoich tabelach,
- preferencje użytkownika wydzielono do `user_profile`, bo nie zależą od danych logowania,
- zgromadzona kwota celu oszczędnościowego nie jest zapisana, lecz wyliczana z wpłat.

Spójność zależności między tabelami, której nie da się wyrazić przez `CHECK` w SQLite (zgodność typu kategorii z typem transakcji, przynależność konta i kategorii do tego samego gospodarstwa), zapewniają wyzwalacze (rozdz. 3.7).

3.6 Indeksy

Oprócz indeksów tworzonych automatycznie dla kluczy głównych i ograniczeń `UNIQUE` zaplanowano indeksy wspierające raporty (filtrowanie po koncie lub kategorii i zakresie dat) oraz generowanie transakcji cyklicznych.

3.7 Widoki, wyzwalacze i procedury

Widoki. Trzy widoki kapsułkują złożone agregacje używane w wielu miejscach aplikacji: `v_budget_usage` (wykorzystanie limitów i status: ok, warning, exceeded), `v_goal_progress` (zgromadzona i brakująca kwota oraz procent realizacji celu) i `v_account_balance_check` (porównanie salda zapisanego z wyliczonym). Widoki tworzymy w migracji Django (`RunSQL`) i odczytujemy przez modele niezarządzane (`managed = False`).

Wyzwalacze. Dwa wyzwalacze wymuszają niezmienniki, których nie wyrazi `CHECK` (nie może on odwoływać się do innych tabel). Analogiczne wyzwalacze `BEFORE UPDATE OF category_id, account_id, target_account_id, tx_type` chronią operację edycji.

Procedury. SQLite nie obsługuje procedur składowanych. Ich funkcję pełnią funkcje warstwy serwisowej Django, z których każda wykonuje całą operację w jednej transakcji bazodanowej (`transaction.atomic`):

Funkcja serwisowa	Odpowiada	Zadanie
`add_transaction`	T1	zapis transakcji, korekta salda, tagi, zwrot ostrzeżenia o limicie
`update_transaction`	T2	cofnięcie starego wpływu, zapis, zastosowanie nowego
`delete_transaction`	T3	usunięcie logiczne i cofnięcie wpływu na saldo (także dla przelewów)
`transfer_between_accounts`	T4	przelew atomowy
`invite_member`, `accept_invitation`, `remove_member`	T5	zarządzanie członkostwem z kontrolą roli
`set_monthly_limit`	T6	upsert limitu
`generate_recurring_transactions`	T7	generowanie transakcji z reguł (polecenie `manage.py`)
`add_goal_contribution`	T8	wpłata i zmiana statusu celu

Przykład funkcji dla T1 (szkic):

```
@transaction.atomic
def add_transaction(*, user, account, category, tx_type, amount_gr, tx_date, description="", tags
    =()):
    assert_member(user, account.household_id)  # autoryzacja
    tx = Transaction.objects.create(account=account, category=category, created_by=user,tx_type=
    tx_type, amount_gr=amount_gr,tx_date=tx_date, description=description)
    delta = amount_gr if tx_type == "income" else -amount_gr
    Account.objects.filter(pk=account.pk).update(balance_gr=F("balance_gr") + delta, updated_at=
    timezone.now())
    tx.tags.set(tags)
    return tx, budget_warning(category, tx_date)  # zapytanie do v_budget_usage
```

4 Transakcje i zapytania SQL

Rozdział opisuje operacje bazodanowe zaplanowane w aplikacji. Wszystkie instrukcje zostały wykonane na bazie utworzonej ze skryptu z załącznika A i danych testowych. Test obejmował m.in. brak zmiany salda przy powtórnym usunięciu transakcji, odrzucenie przelewu na to samo konto lub do innego gospodarstwa, odrzucenie kategorii niezgodnej z typem oraz zgodność sald zapisanych z wyliczonymi po każdej operacji.

4.1 Transakcje modyfikujące dane

Operacje z `BEGIN IMMEDIATE ... COMMIT` wykonywane są atomowo. W razie błędu warstwa serwisowa wykonuje `ROLLBACK`, więc nie powstaje stan częściowy (np. transakcja bez zmiany salda).

4.1.1 T1 – Rejestracja wydatku lub przychodu

Powiązane wymagania: HU.01, HU.02, WB.01, WB.02, WB.04.
Dodanie transakcji i jednoczesna korekta salda konta. Dla przychodu saldo rośnie, dla wydatku maleje. Obie zmiany zachodzą w jednej transakcji bazodanowej. Opcjonalnie przypisywane są tagi.

4.1.2 T2 – Edycja transakcji

Powiązane wymagania: WB.04.
Zmiana kwoty, konta, kategorii lub daty istniejącej transakcji. Najpierw cofany jest wpływ starej wersji na saldo, potem zapisywana nowa wersja i stosowany jej wpływ. Zmiana typu transakcji nie jest dozwolona.

4.1.3 T3 – Usunięcie transakcji (usunięcie logiczne)

Powiązane wymagania: WB.04.
Transakcja nie jest kasowana fizycznie, lecz oznaczana jako nieaktywna (`is_active = 0`), a jej wpływ na saldo jest cofany. Ponowne wywołanie dla tej samej transakcji nie zmienia salda. Poniższy SQL dotyczy przychodu i wydatku. Dla przelewu warstwa serwisowa cofa wpływ analogicznie na oba konta.

4.1.4 T4 – Przelew między kontami

Powiązane wymagania: HU.05, WB.09.
Zmniejszenie salda konta źródłowego i zwiększenie salda docelowego w jednej transakcji bazodanowej. Ograniczenie `CHECK` gwarantuje różne konta i brak kategorii, a wyzwalacz `trg_tx_same_household` – przynależność obu kont do jednego gospodarstwa. Przelew nie jest wliczany do przychodów i wydatków w raportach.

4.1.5 T5 – Zaproszenie, akceptacja i usunięcie członka gospodarstwa

Powiązane wymagania: HU.03, WB.03.
Trzy osobne operacje atomowe. Zaprosić i usunąć członka może wyłącznie aktywny administrator gospodarstwa (warunek `EXISTS`). Nie można usunąć ostatniego administratora.

4.1.6 T6 – Ustawienie lub zmiana limitu miesięcznego

Powiązane wymagania: HU.04, WB.05.
Operacja typu upsert: jeśli dla pary kategoria–miesiąc limit już istnieje, zostaje zaktualizowany (`UNIQUE (category_id, month)`), w przeciwnym razie tworzony.

4.1.7 T7 – Generowanie transakcji cyklicznych

Powiązane wymagania: HU.06, WB.08.
Uruchamiana codziennie przez polecenie zarządzające Django (harmonogram systemowy) oraz przy starcie aplikacji (nadrabianie zaległości). Dla każdej wymagalnej reguły i każdej zaległej daty wstawiana jest transakcja; `UNIQUE (recurring_rule_id, tx_date)` zapewnia idempotentność. Kolejny termin wylicza warstwa serwisowa (arytmetyka dat z obsługą krótszych miesięcy).

4.1.8 T8 – Wpłata na cel oszczędnościowy

Powiązane wymagania: HU.07, WB.10.
Dodanie wpłaty i automatyczna zmiana statusu celu na „zrealizowany", gdy suma wpłat osiągnie kwotę docelową.

4.2 Zapytania odczytujące (zaawansowane)

Poniższe zapytania wykorzystują złączenia wielu tabel, grupowanie i agregację, funkcje okna (`SUM() OVER`, `LAG`), wyrażenia tablicowe (`WITH`) oraz widoki.

4.2.1 Q1 – Wydatki wg kategorii w wybranym okresie

Powiązane wymagania: WB.06, HU.08.
JOIN trzech tabel, filtrowanie po okresie, grupowanie i agregacja. Przelewy są pomijane. Okres zapisany jako przedział półotwarty, co pozwala wykorzystać indeks `(category_id, tx_date)` i `(account_id, tx_date)`.

4.2.2 Q2 – Plan a wykonanie: kategorie w strefie ostrzeżenia lub po przekroczeniu limitu

Powiązane wymagania: HU.04, HU.08, WB.07.
`LEFT JOIN` limitu z wydatkami miesiąca, `GROUP BY` oraz `HAVING` wybierający tylko kategorie, w których wykorzystanie przekroczyło próg ostrzeżenia.

4.2.3 Q3 – Bilans miesiąca

Powiązane wymagania: HU.08, WB.06.
Jedno przejście po danych daje sumę przychodów, wydatków i bilans. Przelewy są wykluczone, więc nie zniekształcają bilansu.

4.2.4 Q4 – Wydatki wg osoby z udziałem procentowym (funkcja okna)

Powiązane wymagania: HU.08, WB.06.
Grupowanie po użytkowniku, który wprowadził transakcję, oraz udział w sumie wydatków gospodarstwa wyliczany funkcją okna `SUM() OVER ()`.

4.2.5 Q5 – Trend miesięczny wydatków (CTE i funkcja LAG)

Powiązane wymagania: HU.08.
Wspólne wyrażenie tablicowe agreguje wydatki na miesiąc, a `LAG()` porównuje każdy miesiąc z poprzednim.

4.2.6 Q6 – Transakcje z tagami

Powiązane wymagania: WB.11, WB.12.
Złączenie przez tabelę n:m i scalenie tagów w jedną kolumnę funkcją `GROUP_CONCAT`. Wynik jest eksportowany do CSV.

4.2.7 Q7 – Widoki: wykorzystanie limitów, postęp celów, weryfikacja sald

Powiązane wymagania: HU.04, HU.07, WB.04.
Zapytania do widoków zdefiniowanych w rozdz. 3.7.

5 Prognoza charakteru encji

Założenia obciążenia. Do szacunków przyjęto docelowo ok. 10 gospodarstw (średnio 2,5 użytkownika), okres 5 lat oraz ok. 100 transakcji miesięcznie na gospodarstwo (stąd ok. 1 200 transakcji rocznie na gospodarstwo i ok. 60 tys. wszystkich wierszy po 5 latach).

Tabela	Sposób użycia	Zmienność	Liczba wierszy
`app_user`	odczyt (logowanie, wyświetlanie)	rzadkie dodania, edycje profilu sporadyczne	ok. 25
`user_profile`	odczyt	jak `app_user`, edycje rzadkie	ok. 25
`household`	odczyt	bardzo rzadko zmieniana	ok. 10
`household_member`	odczyt, rzadki zapis	zaproszenia i zmiany ról sporadyczne	ok. 25
`account`	odczyt i częsty zapis salda	dodawanie rzadkie, ale kolumna `balance_gr` zmienia się przy każdej transakcji	ok. 40
`category`	odczyt (słownik)	dodawanie i edycje rzadkie	ok. 300
`fin_transaction`	zapis (dodawanie) i odczyt (raporty)	intensywne dodawanie (ok. 100/mies. na gospodarstwo), edycje i usunięcia logiczne rzadkie (ok. 5%), brak usuwania fizycznego	ok. 60 tys.
`tag`	odczyt	rzadkie dodania	ok. 100
`fin_transaction_tag`	odczyt i zapis przy transakcji	dodawanie razem z transakcją, część transakcji nieoznaczona tagami	ok. 30 tys.
`monthly_limit`	odczyt (raporty, ostrzeżenia)	ustawiane raz w miesiącu, korekty sporadyczne	ok. 6 000
`recurring_rule`	odczyt (codzienne generowanie)	rzadkie dodania i zmiany statusu, `next_run_date` zmienia się przy każdym generowaniu	ok. 100
`savings_goal`	odczyt	rzadkie dodania, zmiana statusu raz na cel	ok. 20
`goal_contribution`	zapis dopisujący, odczyt (sumy)	dodawanie okresowe, edycje i usunięcia rzadkie	ok. 500

Wnioski projektowe. Dominującą tabelą jest `fin_transaction`, dlatego to na niej koncentrują się indeksy złożone. Obciążenie ma charakter odczyt-intensywny dla raportów i zapis dopisujący dla transakcji, bez masowych modyfikacji historii. Usunięcie logiczne zapobiega fragmentacji i utracie historii.

6 Mapowanie wymagań biznesowych na wymagania funkcjonalne i niefunkcjonalne

Wymagania biznesowe z Etapu 1 (WB) zostały uszczegółowione do wymagań funkcjonalnych (WF) i niefunkcjonalnych (WNF). WB.14 przekłada się na ograniczenie projektowe WNF.07. Pozostałe wymagania WNF zebrano pod tabelą mapowania.

WB	Wymaganie	Treść
WB.01	WF.01	System musi umożliwiać dodanie przychodu lub wydatku z kwotą, datą, kategorią, kontem oraz opcjonalnym opisem i tagami.
	WF.02	Formularz transakcji musi domyślnie podpowiadać bieżącą datę i ostatnio użyte konto, aby skrócić wprowadzanie.
	WNF.01	Dodanie transakcji wymaga wypełnienia co najwyżej 5 pól obowiązkowych (typ, kwota, data, kategoria, konto).
	WNF.02	Czas odpowiedzi API przy zapisie transakcji nie przekracza 300 ms (percentyl 95) przy 60 tys. transakcji w bazie.
WB.02	WF.03	Każda transakcja musi mieć konto. Przychód i wydatek muszą mieć kategorię zgodnego typu (przychód/wydatek).
	WF.04	System musi umożliwiać zarządzanie kategoriami (dodawanie, zmiana nazwy, dezaktywacja) w obrębie gospodarstwa.
	WF.05	System musi umożliwiać zarządzanie kontami (dodawanie, edycja, dezaktywacja, saldo początkowe, typ konta).
WB.03	WF.06	Administrator gospodarstwa musi móc zaprosić użytkownika (po loginie lub adresie e-mail).
	WF.07	Zaproszony użytkownik musi móc zaakceptować zaproszenie. Po akceptacji widzi wspólne konta i kategorie.
	WF.08	Tylko administrator może usuwać członków gospodarstwa. Nie można usunąć ostatniego administratora.
	WNF.03	Użytkownik widzi i modyfikuje wyłącznie dane gospodarstw, w których jest aktywnym członkiem (kontrola autoryzacji w każdym punkcie końcowym API).
	WNF.04	Hasła są przechowywane jako skróty (PBKDF2/Argon2), a dostęp do API chroniony tokenami JWT o krótkim czasie życia.
WB.04	WF.09	Saldo konta musi być aktualizowane automatycznie i atomowo po dodaniu, edycji lub usunięciu transakcji.
	WF.10	System musi udostępniać kontrolę spójności sald (porównanie salda zapisanego z wyliczonym z transakcji).
	WNF.05	Zmiana transakcji i zmiana salda wykonują się w jednej transakcji bazodanowej (ACID). W razie błędu nie powstaje zapis częściowy.
WB.05	WF.11	System musi umożliwiać ustalenie i zmianę miesięcznego limitu dla kategorii wydatków (jeden limit na kategorię i miesiąc).
	WF.12	Limit musi mieć konfigurowalny próg ostrzeżenia (domyślnie 80%).
WB.06	WF.13	System musi prezentować zestawienie wydatków wg kategorii dla wybranego okresu.
	WF.27	Raport miesięczny pokazuje sumy dla kategorii, bilans (przychody minus wydatki) i porównanie z limitem, z możliwością zawężenia do członka lub konta (HU.08).
WB.07	WF.15	System wylicza status limitu (w normie, ostrzeżenie, przekroczony) i wyraźnie oznacza kategorie z przekroczonym limitem.
WB.08	WF.17	System musi umożliwiać zdefiniowanie reguły cyklicznej (kwota, kategoria, konto, częstotliwość, dzień miesiąca, daty początku i końca).

WB	Wymaganie	Treść
	WF.18	System musi umożliwiać wstrzymanie, wznowienie i dezaktywację reguły.
	WF.19	System generuje transakcje z reguł automatycznie (codzienne uruchomienie i nadrabianie zaległości). Generowanie jest idempotentne.
WB.09	WF.20	Przelew między kontami zmienia salda konta źródłowego i docelowego atomowo. Konta muszą być różne i należeć do tego samego gospodarstwa.
	WF.21	Przelewy nie są wliczane do przychodów i wydatków w raportach.
WB.10	WF.22	System musi umożliwiać utworzenie celu oszczędnościowego (nazwa, kwota docelowa, termin).
	WF.23	System musi umożliwiać wpłaty na cel.
	WF.24	Widok celu pokazuje zgromadzoną kwotę, brakującą kwotę i procent realizacji. Po osiągnięciu kwoty docelowej cel otrzymuje status „zrealizowany”.
WB.11	WF.25	System musi umożliwiać tworzenie tagów, przypisywanie wielu tagów do transakcji i filtrowanie listy transakcji po tagu.
WB.12	WF.26	System musi umożliwiać eksport transakcji z wybranego okresu do pliku CSV.
WB.14	WNF.07	Każde gospodarstwo ma jedną walutę (`household.currency_code`, domyślnie PLN). System nie obsługuje kursów walut.

Pozostałe wymagania niefunkcjonalne

WNF.09 API REST jest udokumentowane w standardzie OpenAPI i dostępne w Swagger UI.

WNF.10 Warstwa serwisowa ma pokrycie testami automatycznymi co najmniej 70%. Kluczowe scenariusze interfejsu są pokryte testami Playwright.

WNF.11 System obsługuje do 10 gospodarstw i 25 użytkowników na jednej instancji SQLite (tryb WAL).

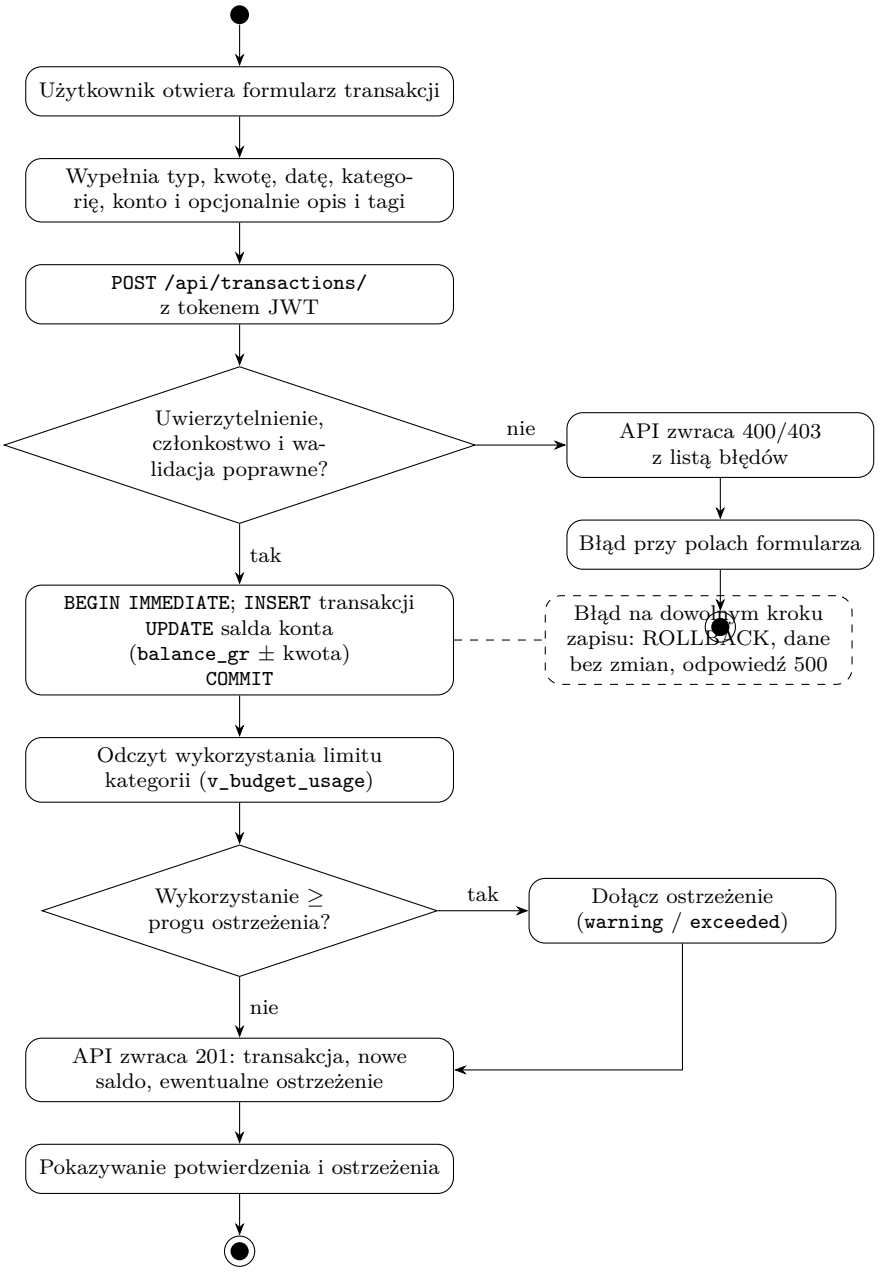
WNF.12 Dane można zarchiwizować kopią pliku bazy (polecenie `.backup` SQLite). Opisana jest procedura odtworzenia.

WNF.13 Wszystkie błędy walidacji zwracane są w formacie JSON z czytelnym komunikatem, a interfejs wyświetla je przy polach formularza.

7 Diagramy UML

7.1 Diagram aktywności: rejestracja wydatku

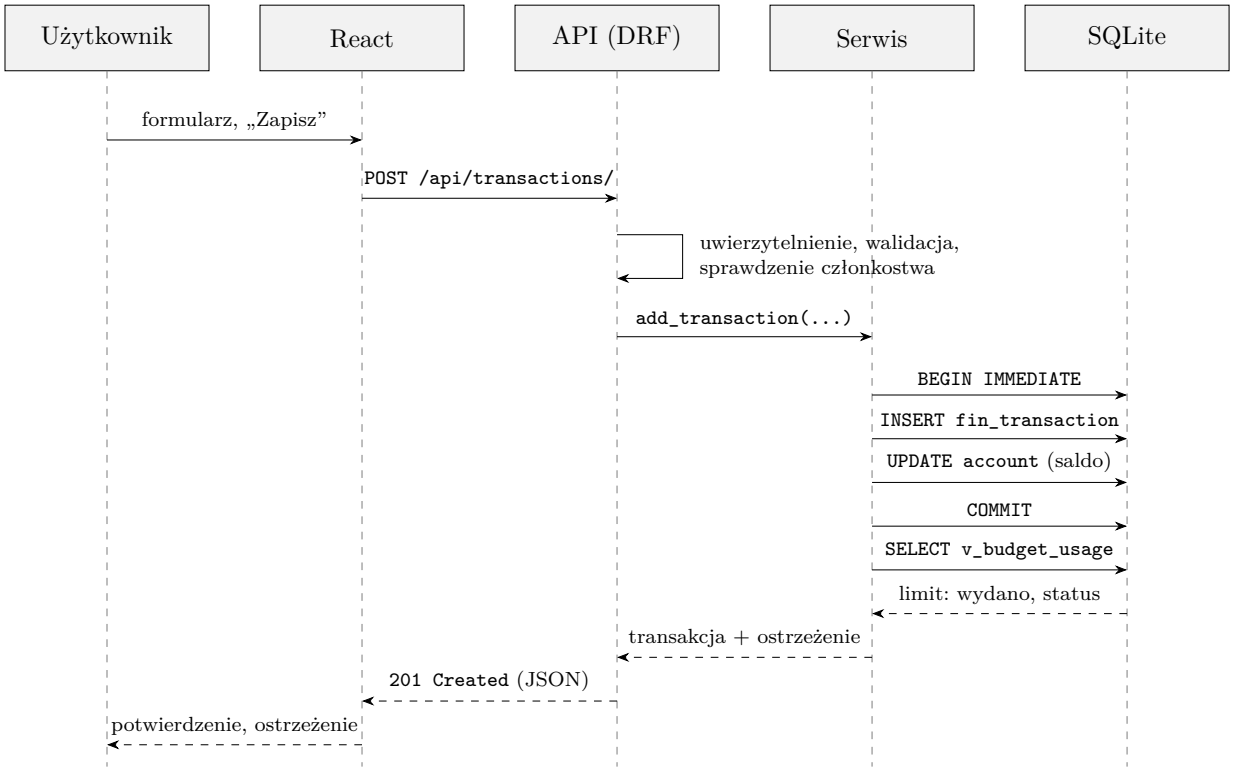
Diagram przedstawia logikę przypadku UC.04 (HU.01, WB.01, WB.04, WB.07): od wypełnienia formularza, przez walidację i zapis transakcji ze zmianą salda, po wyliczenie ostrzeżenia o limicie.



Rysunek 3: Diagram aktywności rejestracji wydatku

7.2 Diagram sekwencji: rejestracja wydatku

Diagram pokazuje interakcję elementów systemu w scenariuszu UC.04: przeglądarka, API, warstwa serwisowa i baza SQLite.



Rysunek 4: Diagram sekwencji rejestracji wydatku. Przy błędzie walidacji API zwraca 400/403 bez wywołania serwisu, a przy błędzie zapisu serwis wykonuje ROLLBACK i API zwraca 500.

8 Opisy przypadków użycia

Poniżej opisano najważniejsze przypadki użycia (aktorzy, warunki, ścieżka podstawowa i ścieżki alternatywne). Pozostałe (UC.01–UC.03, UC.11, UC.13) mają charakter standardowych operacji zarządzania danymi (CRUD) i wynikają bezpośrednio z wymagań WF.03–WF.05, WF.08, WF.09 i WF.26.

8.1 UC.04 Rejestracja transakcji (przychód/wydatek)

Aktor	Domownik.
Powiązania	HU.01, HU.02; WB.01, WB.02, WB.04, WB.07; WF.01–WF.03, WF.10, WF.16; T1; Q7.
Warunki wstępne	Użytkownik jest zalogowany i jest aktywnym członkiem gospodarstwa. Istnieje co najmniej jedno konto i kategoria odpowiedniego typu.
Ścieżka podstawowa	1. Domownik otwiera formularz i wybiera typ (przychód lub wydatek). 2. Podaje kwotę, datę, kategorię, konto oraz opcjonalnie opis i tagi. 3. System waliduje dane (kwota > 0, data niepusta, kategoria zgodna z typem, konto i kategoria z tego samego gospodarstwa). 4. System zapisuje transakcję i zmienia saldo konta w jednej transakcji bazodanowej. 5. System sprawdza wykorzystanie limitu kategorii i zwraca potwierdzenie. 6. Transakcja jest widoczna na liście transakcji gospodarstwa.
Ścieżki alternatywne	3a. Dane niepoprawne: system zwraca błędy walidacji, nic nie zapisuje, formularz pozostaje wypełniony. 3b. Kategoria lub konto z innego gospodarstwa: odrzucenie (403/400). 4a. Błąd zapisu: ROLLBACK, komunikat o błędzie, saldo bez zmian. 5a. Wykorzystanie osiągnęło próg (domyślnie 80%) lub 100%: odpowiedź zawiera ostrzeżenie, a kategoria jest oznaczona.
Warunki końcowe	Saldo konta zmniejszone (wydatek) lub zwiększone (przychód) o kwotę. Transakcja widoczna w raportach.

8.2 UC.05 Przelew między kontami

Aktor	Domownik.
Powiązania	HU.05; WB.09; WF.20, WF.21; T4.
Warunki wstępne	Użytkownik jest członkiem gospodarstwa. W gospodarstwie istnieją co najmniej dwa konta.
Ścieżka podstawowa	1. Domownik wybiera konto źródłowe, konto docelowe, kwotę i datę. 2. System sprawdza, że konta są różne i należą do tego samego gospodarstwa. 3. System zapisuje przelew, zmniejsza saldo konta źródłowego i zwiększa saldo docelowego w jednej transakcji bazodanowej. 4. System potwierdza operację i pokazuje nowe salda.
Ścieżki alternatywne	2a. To samo konto lub konto z innego gospodarstwa: odrzucenie z komunikatem. 3a. Błąd zapisu: ROLLBACK, oba salda bez zmian.
Warunki końcowe	Salda obu kont zmienione o kwotę przelewu. Przelew nie jest wliczany do przychodów ani wydatków w raportach.

8.3 UC.06 Ustalanie limitu miesięcznego

Aktor	Domownik.
Powiązania	HU.04; WB.05, WB.07; WF.12, WF.13, WF.15; T6; Q2, Q7.
Warunki wstępne	Użytkownik jest członkiem gospodarstwa. Istnieje kategoria typu „wydatek”.
Ścieżka podstawowa	1. Domownik wybiera kategorię i miesiąc oraz podaje kwotę limitu i opcjonalnie próg ostrzeżenia. 2. System zapisuje limit (nowy lub nadpisany dla pary kategoria–miesiąc). 3. Widok budżetu pokazuje kwotę wykorzystaną, pozostałą i procent wykorzystania.
Ścieżki alternatywne	1a. Kwota ≤ 0 lub próg poza zakresem 1–100: błąd walidacji. 1b. Kategoria typu „przychód”: odrzucenie. 3a. Po przekroczeniu 100% kategoria jest wyraźnie oznaczona jako przekroczona.
Warunki końcowe	Dla kategorii i miesiąca istnieje dokładnie jeden aktywny limit.

8.4 UC.07 Zarządzanie regułami cyklicznymi

Aktor	Domownik.
Powiązania	HU.06; WB.08; WF.17, WF.18.
Warunki wstępne	Użytkownik jest członkiem gospodarstwa. Istnieje konto i kategoria.
Ścieżka podstawowa	1. Domownik definiuje regułę: typ, kwota, kategoria, konto, częstotliwość, dzień miesiąca, data początku i opcjonalnie końca. 2. System zapisuje regułę ze statusem „aktywna” i wylicza termin pierwszego wystąpienia. 3. Domownik może regułę wstrzymać, wznowić lub dezaktywować.
Ścieżki alternatywne	1a. Data końca wcześniejsza niż początku lub brak dnia miesiąca dla częstotliwości miesięcznej: błąd walidacji. 3a. Reguła wstrzymana nie generuje transakcji do czasu wznowienia. Po wznowieniu zaległe terminy są nadrabiane zgodnie z UC.08.
Warunki końcowe	Reguła w jednym ze stanów: aktywna, wstrzymana, nieaktywna (rys. 5a).

8.5 UC.08 Generowanie transakcji cyklicznych

Aktor	Harmonogram (aktor systemowy).
Powiązania	HU.06; WB.08; WF.19; T7.
Warunki wstępne	Istnieją aktywne reguły z terminem `next_run_date` nie późniejszym niż dzisiaj.
Ścieżka podstawowa	1. Harmonogram uruchamia polecenie `manage.py generate_recurring` (codziennie oraz przy starcie aplikacji). 2. System wybiera reguły wymagalne (aktywne, niewstrzymane, z terminem $\leq$ dziś i przed datą zakończenia). 3. Dla każdej reguły i każdej zaległej daty system tworzy transakcję i koryguje saldo konta, a następnie przesuwa termin kolejnego wystąpienia. 4. Reguły po dacie zakończenia są dezaktywowane.
Ścieżki alternatywne	3a. Transakcja dla danej reguły i daty już istnieje: pominięcie bez zmiany salda (idempotentność dzięki `UNIQUE (recurring_rule_id, tx_date)`). 3b. Błąd przy jednej regule: ROLLBACK tylko tej reguły, pozostałe są przetwarzane dalej, błąd trafia do logu.
Warunki końcowe	Wszystkie wymagalne transakcje wygenerowane dokładnie raz, salda skorygowane.

8.6 UC.09 Cel oszczędnościowy i wpłaty

Aktor	Domownik.
Powiązania	HU.07; WB.10; WF.22–WF.24; T8; Q7.
Warunki wstępne	Użytkownik jest członkiem gospodarstwa.
Ścieżka podstawowa	1. Domownik tworzy cel: nazwa, kwota docelowa, termin. 2. Domownik dodaje wpłaty na cel (kwota, data, notatka). 3. System po każdej wpłacie wylicza zgromadzoną kwotę, brakującą kwotę i procent realizacji. 4. Po osiągnięciu kwoty docelowej cel otrzymuje status „zrealizowany”.
Ścieżki alternatywne	1a. Kwota docelowa ≤ 0 lub brak terminu: błąd walidacji. 2a. Wpłata na cel zrealizowany lub anulowany: odrzucenie. *a. Domownik może anulować cel (status „anulowany”).
Warunki końcowe	Cel w stanie: w toku, zrealizowany lub anulowany (rys. 5b).

8.7 UC.10 Raport miesięczny

Aktor	Domownik.
Powiązania	HU.08; WB.06, WB.07; WF.14, WF.21, WF.27; Q1–Q5.
Warunki wstępne	Użytkownik jest członkiem gospodarstwa.
Ścieżka podstawowa	1. Domownik wybiera miesiąc (lub dowolny okres) i opcjonalnie filtr: członek gospodarstwa, konto. 2. System wylicza sumy wydatków i przychodów wg kategorii, bilans miesiąca (przychody minus wydatki) oraz porównanie z limitami. 3. System prezentuje zestawienie tabelą i wykresem, kategorie z przekroczonym limitem są oznaczone.
Ścieżki alternatywne	2a. Brak transakcji w okresie: raport pustych wartości z komunikatem. 2b. Brak limitu dla kategorii: kolumna porównania pozostaje pusta.
Warunki końcowe	Raport nie zmienia danych. Przelewy nie są wliczane do przychodów i wydatków.

8.8 UC.11 Zapraszanie członków do gospodarstwa

Aktor	Administrator gospodarstwa (oraz zapraszany użytkownik).
Powiązania	HU.03; WB.03; WF.06–WF.08; T5.
Warunki wstępne	Zapraszający jest aktywnym administratorem gospodarstwa. Zapraszany użytkownik ma konto w systemie.
Ścieżka podstawowa	1. Administrator podaje login (lub e-mail) zapraszanego użytkownika. 2. System tworzy członkostwo o statusie „zaproszony”. 3. Zaproszony akceptuje zaproszenie. 4. System zmienia status na „aktywny” i zapisuje datę dołączenia. Użytkownik widzi wspólne konta i kategorie.
Ścieżki alternatywne	2a. Użytkownik nie istnieje lub już jest członkiem: komunikat o błędzie. 2b. Zapraszający nie jest administratorem: odmowa (403). 3a. Zaproszony odrzuca zaproszenie: członkostwo dezaktywowane. *a. Administrator usuwa członka (UC.13): tylko administrator, nigdy ostatni administrator.
Warunki końcowe	Użytkownik jest aktywnym członkiem gospodarstwa z rolą „zwykły” (rys. 5c).

9 Podsumowanie

W Etapie 2 określono technologie, zaprojektowano bazę danych składającą się z 13 tabel z relacjami 1:1, 1:n i n:m, pełnymi więzami, polami audytowymi, widokami i wyzwalaczami, uzasadniono schemat w 3NF wraz z jednym świadomym wyjątkiem (saldo konta), opisano osiem transakcji bazodanowych i siedem zapytań odczytujących wraz z kodem SQL sprawdzonym na SQLite, oszacowano charakter encji oraz przygotowano materiały analityczne: mapowanie WB → WF/WNF, diagramy UML i opisy przypadków użycia.